

```
In [1]: import seaborn as sns
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.impute import KNNImputer
from thefuzz import process
from sklearn.preprocessing import LabelEncoder
from sklearn.preprocessing import MinMaxScaler, StandardScaler, RobustScaler
```

```
In [2]: df=sns.load_dataset('titanic')
print("head: ",df.head())
print("info",df.info())
print("dim",df.shape)
print("nill",df.isnull().sum())
print("describe",df.describe())
```

head:	survived	pclass	sex	age	sibsp	parch	fare	embarked
0	0	3	male	22.0	1	0	7.2500	S Third
1	1	1	female	38.0	1	0	71.2833	C First
2	1	3	female	26.0	0	0	7.9250	S Third
3	1	1	female	35.0	1	0	53.1000	S First
4	0	3	male	35.0	0	0	8.0500	S Third

	who	adult_male	deck	embark_town	alive	alone
0	man	True	NaN	Southampton	no	False
1	woman	False	C	Cherbourg	yes	False
2	woman	False	NaN	Southampton	yes	True
3	woman	False	C	Southampton	yes	False
4	man	True	NaN	Southampton	no	True

<class 'pandas.core.frame.DataFrame'>

RangeIndex: 891 entries, 0 to 890

Data columns (total 15 columns):

#	Column	Non-Null Count	Dtype
0	survived	891 non-null	int64
1	pclass	891 non-null	int64
2	sex	891 non-null	object
3	age	714 non-null	float64
4	sibsp	891 non-null	int64
5	parch	891 non-null	int64
6	fare	891 non-null	float64
7	embarked	889 non-null	object
8	class	891 non-null	category
9	who	891 non-null	object
10	adult_male	891 non-null	bool
11	deck	203 non-null	category
12	embark_town	889 non-null	object
13	alive	891 non-null	object
14	alone	891 non-null	bool

dtypes: bool(2), category(2), float64(2), int64(4), object(5)

memory usage: 80.7+ KB

info None

dim (891, 15)

nill survived 0

pclass 0

sex 0

age 177

sibsp 0

parch 0

fare 0

embarked 2

class 0

who 0

adult_male 0

deck 688

embark_town 2

alive 0

alone 0

dtype: int64

describe	survived	pclass	age	sibsp	parch
count	891.000000	891.000000	714.000000	891.000000	891.000000
mean	0.383838	2.308642	29.699118	0.523008	0.381594
208					

std	0.486592	0.836071	14.526497	1.102743	0.806057	49.693
429						
min	0.000000	1.000000	0.420000	0.000000	0.000000	0.000
000						
25%	0.000000	2.000000	20.125000	0.000000	0.000000	7.910
400						
50%	0.000000	3.000000	28.000000	0.000000	0.000000	14.454
200						
75%	1.000000	3.000000	38.000000	1.000000	0.000000	31.000
000						
max	1.000000	3.000000	80.000000	8.000000	6.000000	512.329
200						

create problem

```
In [3]: df.loc[0, 'embark_town'] = 'southampton'
df.loc[1, 'embark_town'] = 'SOUTHAMPTON'
df.loc[3, 'embark_town'] = 'Southampton'
print(df['embark_town'].unique())
```

['southampton' 'SOUTHAMPTON' 'Southampton' 'Queenstown' 'Cherbourg' nan]

solving it

```
In [4]: df['embark_town'] = df['embark_town'].str.title()
print(df['embark_town'].unique())
```

['Southampton' 'Queenstown' 'Cherbourg' nan]

checking the null

```
In [5]: print(df['embark_town'].isnull().sum())
```

2

filling the missing values

```
In [6]: df['embark_town'] = df['embark_town'].fillna('kingston')
print(df['embark_town'].unique())
```

['Southampton' 'Queenstown' 'Cherbourg' 'kingston']

duplicate

```
In [7]: dr = df.iloc[:5]
df_dup = pd.concat([df, dr])
print(df_dup.head(3))
print(df_dup.shape)
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class
0	0	3	male	22.0	1	0	7.2500	S	Third
1	1	1	female	38.0	1	0	71.2833	C	First
2	1	3	female	26.0	0	0	7.9250	S	Third

	who	adult_male	deck	embark_town	alive	alone
0	man	True	NaN	Southampton	no	False
1	woman	False	C	Southampton	yes	False
2	woman	False	NaN	Southampton	yes	True

(896, 15)

```
In [8]: print(df_dup.duplicated().sum())
```

112

Find duplicated

```
In [9]: print(df_dup.duplicated())
```

```
0    False
1    False
2    False
3    False
4    False
...
0     True
1     True
2     True
3     True
4     True
Length: 896, dtype: bool
```

```
In [10]: print(df_dup.drop_duplicates())
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class
\									
0	0	3	male	22.0	1	0	7.2500	S	Third
1	1	1	female	38.0	1	0	71.2833	C	First
2	1	3	female	26.0	0	0	7.9250	S	Third
3	1	1	female	35.0	1	0	53.1000	S	First
4	0	3	male	35.0	0	0	8.0500	S	Third
..	...	...	...	...	...	...	...	...	...
885	0	3	female	39.0	0	5	29.1250	Q	Third
887	1	1	female	19.0	0	0	30.0000	S	First
888	0	3	female	NaN	1	2	23.4500	S	Third
889	1	1	male	26.0	0	0	30.0000	C	First
890	0	3	male	32.0	0	0	7.7500	Q	Third
	who	adult_male	deck	embark_town	alive	alone			
0	man	True	NaN	Southampton	no	False			
1	woman	False	C	Southampton	yes	False			
2	woman	False	NaN	Southampton	yes	True			
3	woman	False	C	Southampton	yes	False			
4	man	True	NaN	Southampton	no	True			
..	...	...	...	...	...	...			
885	woman	False	NaN	Queenstown	no	False			
887	woman	False	B	Southampton	yes	True			
888	woman	False	NaN	Southampton	no	False			
889	man	True	C	Cherbourg	yes	True			
890	man	True	NaN	Queenstown	no	True			

[784 rows x 15 columns]

type mistake

```
In [11]: df.loc[0,'embark_town']='southamphton'
df.loc[1,'embark_town']='southamphthon'
```

```
In [12]: print(df['embark_town'].value_counts())
```

```
embark_town
Southampton    643
Cherbourg      167
Queenstown     77
kingston        2
southamphton    1
southamphthon   1
Name: count, dtype: int64
```

```
In [13]: df['embark_town']=df['embark_town'].replace({'southamphton':'Southampton',
                                                    'southamphthon':'Southampton'})
print(df['embark_town'].value_counts())
```

```
embark_town
Southampton    645
Cherbourg      167
Queenstown     77
kingston        2
Name: count, dtype: int64
```

```
In [14]: print(sorted(df['embark_town'].dropna().unique()))

['Cherbourg', 'Queenstown', 'Southampton', 'kingston']
```

```
In [15]: print(df['embark_town'].value_counts())
```

```
embark_town
Southampton    645
Cherbourg      167
Queenstown     77
kingston        2
Name: count, dtype: int64
```

```
In [16]: df.loc[0,'age']=240
df.loc[1,'age']=300
df['age'].describe()
```

```
Out[16]: count    714.000000
mean      30.371387
std       19.362012
min        0.420000
25%       20.125000
50%       28.000000
75%       38.750000
max       300.000000
Name: age, dtype: float64
```

```
In [17]: df.loc[df['age']>120,'age']=(df.loc[df['age']> 120,'age']/12)
df['age'].describe()
```

```
Out[17]: count    714.000000
mean      29.678109
std       14.525902
min        0.420000
25%       20.000000
50%       28.000000
75%       38.000000
max       80.000000
Name: age, dtype: float64
```

mapping the value

```
In [18]: df['survived_label']=df['survived'].map({0:"no",1:"yes"})
print(df['survived_label'].head())
```

```
0    no
1    yes
2    yes
3    yes
4    no
Name: survived_label, dtype: object
```

redundant column meand no useful column (more noise data)

```
In [19]: df.isnull().mean() #checking
```

```
Out[19]: survived      0.000000
pclass      0.000000
sex      0.000000
age      0.198653
sibsp      0.000000
parch      0.000000
fare      0.000000
embarked      0.002245
class      0.000000
who      0.000000
adult_male      0.000000
deck      0.772166
embark_town      0.000000
alive      0.000000
alone      0.000000
survived_label      0.000000
dtype: float64
```

```
df.drop(columns=["deck"],inplace=True)
```

```
In [20]: print(df.head())
```

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class
0	0	3	male	20.0	1	0	7.2500	S	Third
1	1	1	female	25.0	1	0	71.2833	C	First
2	1	3	female	26.0	0	0	7.9250	S	Third
3	1	1	female	35.0	1	0	53.1000	S	First
4	0	3	male	35.0	0	0	8.0500	S	Third

	who	adult_male	deck	embark_town	alive	alone	survived_label
0	man	True	NaN	Southampton	no	False	no
1	woman	False	C	Southampton	yes	False	yes
2	woman	False	NaN	Southampton	yes	True	yes
3	woman	False	C	Southampton	yes	False	yes
4	man	True	NaN	Southampton	no	True	no

```
df['age'].isnull().sum()
```

mean method to fill the null values

```
In [21]: df['age']=df['age'].fillna(df['age'].mean())
df.head(20)
```

Out[21]:

	survived	pclass	sex	age	sibsp	parch	fare	embarked	class
0	0	3	male	20.000000	1	0	7.2500	S	Third
1	1	1	female	25.000000	1	0	71.2833	C	First
2	1	3	female	26.000000	0	0	7.9250	S	Third
3	1	1	female	35.000000	1	0	53.1000	S	First
4	0	3	male	35.000000	0	0	8.0500	S	Third
5	0	3	male	29.678109	0	0	8.4583	Q	Third
6	0	1	male	54.000000	0	0	51.8625	S	First
7	0	3	male	2.000000	3	1	21.0750	S	Third
8	1	3	female	27.000000	0	2	11.1333	S	Third
9	1	2	female	14.000000	1	0	30.0708	C	Second
10	1	3	female	4.000000	1	1	16.7000	S	Third
11	1	1	female	58.000000	0	0	26.5500	S	First
12	0	3	male	20.000000	0	0	8.0500	S	Third
13	0	3	male	39.000000	1	5	31.2750	S	Third
14	0	3	female	14.000000	0	0	7.8542	S	Third
15	1	2	female	55.000000	0	0	16.0000	S	Second
16	0	3	male	2.000000	4	1	29.1250	Q	Third
17	1	2	male	29.678109	0	0	13.0000	S	Second
18	0	3	female	31.000000	1	0	18.0000	S	Third
19	1	3	female	29.678109	0	0	7.2250	C	Third

median-data is nearly normal

```
df['age']=df['age'].fillna(df['age'].median()) df.head(20)
```

Mode - most frequency value

```
df['age']=df['age'].fillna(df['age'].mode()) df.head(20)
```

method 4: Forward Fill using the previous value to fill missing value

```
In [22]: df['embarked_ffill']=df['embarked'].ffill()  
print(df['embarked_ffill'].isnull().sum())
```

0

method 5: KNN imputation fill missing values using information from similar rows instead of using of single value(mead/median) find the the k values to fill the missing values

```
In [23]: import pandas as pd
import numpy as np
data={
    'age': [22,25,np.nan,24,25],
    'fare': [7.25,8.05,7.90,8.00,7.50]
}
df=pd.DataFrame(data)
print(df)
```

	age	fare
0	22.0	7.25
1	25.0	8.05
2	NaN	7.90
3	24.0	8.00
4	25.0	7.50

```
In [24]: knn=KNNImputer(n_neighbors=3)
df_knn=pd.DataFrame(knn.fit_transform(df),columns=df.columns)
print(df_knn)
```

	age	fare
0	22.000000	7.25
1	25.000000	8.05
2	24.666667	7.90
3	24.000000	8.00
4	25.000000	7.50

```
In [25]: import seaborn as sns
from sklearn.impute import KNNImputer
df = sns.load_dataset('titanic')
df['embarked'] = df['embarked'].ffill()
cols = ['age', 'fare', 'sibsp', 'parch', 'pclass']
knn = KNNImputer(n_neighbors=5)
df[cols] = knn.fit_transform(df[cols])
print(df.isnull().sum())
```

survived	0
pclass	0
sex	0
age	0
sibsp	0
parch	0
fare	0
embarked	0
class	0
who	0
adult_male	0
deck	688
embark_town	2
alive	0
alone	0

dtype: int64

```
In [26]: df['embarked'] = df['embarked'].fillna('Unknown')
valid_ports = ['C', 'Q', 'S']
def fuzzy_correct(value):
```



```

    if value == 'Unknown':
        return value
    match, score = process.extractOne(str(value), valid_ports)
    if score >= 80:
        return match
    return value
df['embarked'] = df['embarked'].apply(fuzzy_correct)
cols = ['age', 'fare']
for col in cols:
    df[col] = df[col].fillna(df[col].median())
print(df.isnull().sum())

```

```

survived      0
pclass        0
sex           0
age           0
sibsp         0
parch         0
fare          0
embarked      0
class         0
who           0
adult_male    0
deck          688
embark_town    2
alive         0
alone         0
dtype: int64

```

Backward fill

df.bfill(inplace=True) or df.fillna(method='bfill',inplace=True)

encoding

convert text in number

types of encoding

label encoding(ordinal data)

assign a number to each category

one hot encoding

creates a separate binary column for each category example : country ,color,city ,department

targeting encoding

replace categories with average value of the target variable.

```
In [27]: # label encoding
df=sns.load_dataset('titanic')
le=LabelEncoder()
df['sex_encoder']=le.fit_transform(df['sex'])
# one hot encoding
df=pd.get_dummies(df,columns=['embarked'])
# target encoding
target_map=df.groupby('pclass')['survived'].mean()
df['class_target']=(df['pclass'].map(target_map))
print(df.head())
```

	survived	pclass	sex	age	sibsp	parch	fare	class	who	\
0	0	3	male	22.0	1	0	7.2500	Third	man	
1	1	1	female	38.0	1	0	71.2833	First	woman	
2	1	3	female	26.0	0	0	7.9250	Third	woman	
3	1	1	female	35.0	1	0	53.1000	First	woman	
4	0	3	male	35.0	0	0	8.0500	Third	man	

	adult_male	deck	embark_town	alive	alone	sex_encoder	embarked_C	\
0	True	NaN	Southampton	no	False	1	False	
1	False	C	Cherbourg	yes	False	0	True	
2	False	NaN	Southampton	yes	True	0	False	
3	False	C	Southampton	yes	False	0	False	
4	True	NaN	Southampton	no	True	1	False	

	embarked_Q	embarked_S	class_target
0	False	True	0.242363
1	False	False	0.629630
2	False	True	0.242363
3	False	True	0.629630
4	False	True	0.242363

feature scaling

transform the value into specific range

minmax scaler

scales values based on the data use in nn disadvantage: sensitive to outliers

Standard scaler

centre data around mean = 0 and std = 1 work well for disteibution

disadvange: still affect by extreme outlier

robust

use median and iqr instead of mean and std

handle outlier well better for skew distbution

disadvange

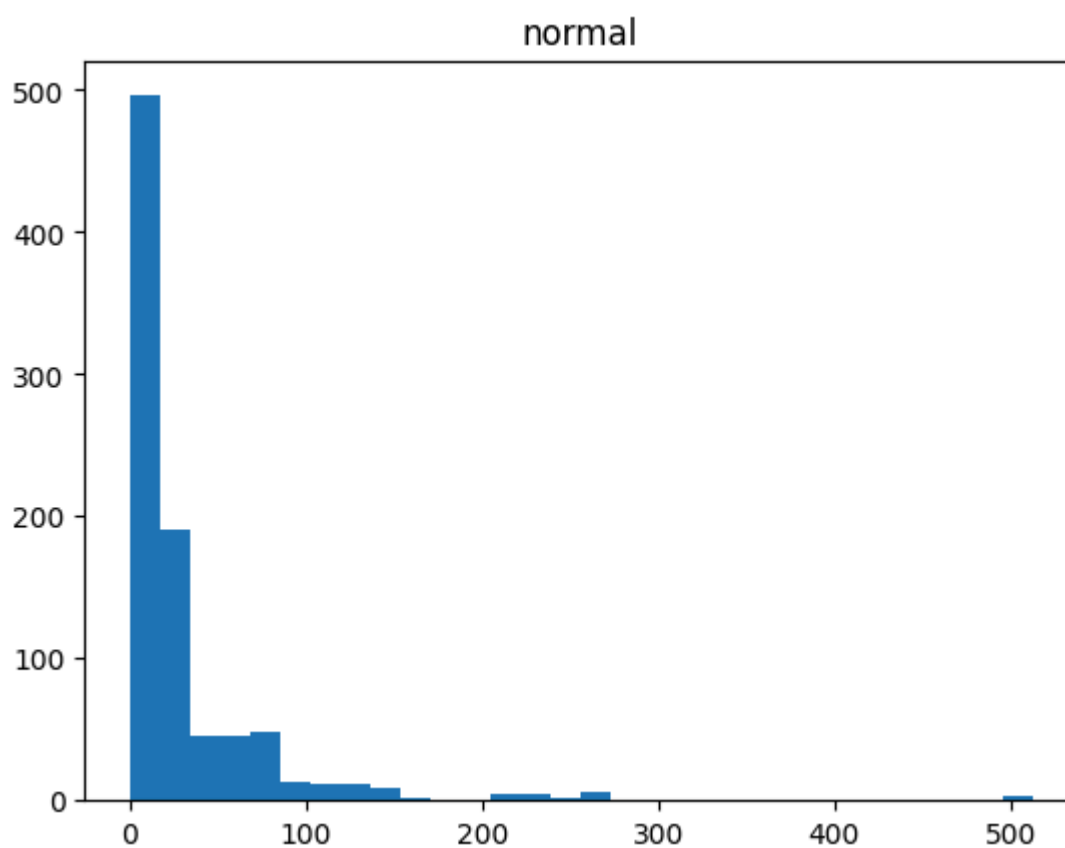
values not into 0 to 1

```
In [28]: s=MinMaxScaler()  
df[['age_mm','fare_mm']]=s.fit_transform(df[['age','fare']])  
df[['age_mm','fare_mm']].head()
```

```
Out[28]:
```

	age_mm	fare_mm
0	0.271174	0.014151
1	0.472229	0.139136
2	0.321438	0.015469
3	0.434531	0.103644
4	0.434531	0.015713

```
In [29]: plt.hist(df['fare'].dropna(),bins=30)  
plt.title("normal")  
plt.show()
```



```
In [30]: s=StandardScaler()  
df[['age_mm','fare_mm']]=s.fit_transform(df[['age','fare']])  
df[['age_mm','fare_mm']].head()
```

Out[30]:

	age_mm	fare_mm
0	-0.530377	-0.502445
1	0.571831	0.786845
2	-0.254825	-0.488854
3	0.365167	0.420730
4	0.365167	-0.486337

```
In [31]: s=RobustScaler()  
df[['age_mm','fare_mm']]=s.fit_transform(df[['age','fare']])  
df[['age_mm','fare_mm']].head()
```

Out[31]:

	age_mm	fare_mm
0	-0.335664	-0.312011
1	0.559441	2.461242
2	-0.111888	-0.282777
3	0.391608	1.673732
4	0.391608	-0.277363